

PROJECT REPORT OF CO-OP Project at Industry (Module-2)

ON

ERRAM: Emotional Reliance Risk Assessment Model

submitted in partial fulfilment of the requirements for the award of degree of

BACHELOR OF ENGINEERING

In

COMPUTER SCIENCE AND ENGINEERING

Submitted by:

Kritika Bansal, Niharika Kapoor

2210990524, 2210991973

Supervised By:

Dr. Lalit K. Sharma

Assistant Professor

Department of Computer Science & Engineering

Chitkara University, Punjab



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

**CHITKARA UNIVERSITY INSTITUTE OF ENGINEERING AND TECHNOLOGY
CHITKARA UNIVERSITY, PUNJAB, INDIA**

CONTENTS

Sr. No.	Title	Page No.
	Acknowledgement	iii
	List of Figures and Tables	iv
	Abstract	v
1	Introduction	1
2	Methodology	2
3	Tools and Technologies	5
4	Implementation	6
5	Major Findings/Outcomes/Output/Results	8
6	Conclusion and Future Scope	10
	References	12
	Appendices	13

DECLARATION

I hereby certify that the work which is being presented in the project report entitled “**ERRAM: Emotional Reliance Risk Assessment Model**” in partial fulfilment of requirement for the award of the degree of Bachelor of Engineering (Computer Science and Engineering) submitted in the department of Computer Science and Engineering at Chitkara University Institute of Engineering and Technology, Chitkara University, Punjab, India, is an authentic record of my own work carried out under the supervision of Dr. Lalit K. Sharma. The matter presented in this project report has not been submitted in any other university/institute for the award of any degree.

Place: Rajpura

Kritika Bansal, Niharika Kapoor

Date: 1,May,2026

(2210990524, 2210991973)

This is to certify that the above statement made by the candidate is correct to the best of my knowledge and belief.

Dr. Lalit K. Sharma

Assistant Professor

Department of Computer Science and Engineering

Chitkara University Institute of Engineering and Technology,

Chitkara University, Punjab, India

ACKNOWLEDGEMENT

We take this opportunity to express our heartfelt gratitude to all those who have supported and guided us in the successful completion of our project, ***ERRAM – Emotional Reliance Risk Assessment Model***.

We are especially grateful to our respected mentor, **Dr. Lalit K. Sharma**, whose invaluable guidance, constructive feedback, and constant encouragement played a vital role throughout the development of this project. His insights greatly helped us refine our approach and strengthen our understanding.

We would also like to thank our faculty and institution for providing us with the platform and resources to work on such an innovative and meaningful project, allowing us to enhance both our technical and problem-solving skills.

Our sincere thanks to our team members for their cooperation, dedication, and shared efforts that made this project a success. Working together has been a rewarding experience.

Finally, we extend our gratitude to our parents and friends for their continuous support, motivation, and belief in us, which kept us going during challenging phases of this project.

Kritika Bansal

2210990524

Niharika Kapoor

2210991973

List of Figures and Tables

Table No.	Title	Page No.
Table 4.1	Frameworks and Technologies Used	6
Table 4.2	Emotion to Risk Mapping	7
Table 5.1	Model Performance Metrics (Validation vs Test)	8
Table 5.2	Sample Predictions (Expected vs Observed)	9
Table A	Sample Inputs and Outputs	13
Table B	Emotion to Risk Mapping (Appendix)	13

ABSTRACT

The rapid adoption of conversational AI systems has transformed the way individuals express emotions and seek support in digital environments. While these platforms offer accessibility and convenience, they lack mechanisms to monitor emotional dependency or detect signs of distress in user interactions.

This project, ERRAM – Emotional Reliance Risk Assessment Model, addresses this gap by providing a web-based solution that analyses user-generated text and classifies emotional risk into three categories: Low, Medium, and High. The system is built using Natural Language Processing (NLP) and Machine Learning techniques, specifically employing TF-IDF vectorization and a Logistic Regression classifier trained on a labelled dataset of over 16,000 sentences.

The application processes user input in real time, generates a risk classification along with confidence scores, and provides a probability distribution across all categories. Additionally, it offers contextual suggestions to guide users based on their emotional state. Designed as a lightweight and interpretable system, ERRAM ensures efficient performance without requiring high computational resources.

The project demonstrates that meaningful emotional risk detection can be achieved using traditional machine learning techniques, contributing towards responsible AI usage and user well-being awareness.

1. Introduction

The widespread adoption of conversational AI platforms has fundamentally altered how individuals articulate emotional experiences in digital environments. Tools designed for task automation and information retrieval are now routinely used as emotional outlets, particularly among students and young adults who find AI systems more accessible and less intimidating than direct human interaction. While this accessibility carries genuine benefits, the absence of built-in monitoring mechanisms means that escalating emotional dependency often goes undetected until it manifests as a significant concern.

ERRAM, the Emotional Reliance Risk Assessment Model, is a web-based application that examines free-text user input and categories the emotional risk present into one of three levels: Low, Medium, or High. The system is built on a supervised machine learning framework, combining Term Frequency-Inverse Document Frequency (TF-IDF) feature extraction with a Logistic Regression classifier trained on over sixteen thousand labelled conversational sentences sourced from a publicly available Kaggle dataset.

The project emerged from a recognition that current conversational AI systems prioritise response generation over user welfare monitoring. A lightweight, interpretable tool capable of running on standard hardware and returning real-time assessments addresses a practical gap in responsible AI deployment. Beyond its technical function, ERRAM serves as an educational instrument, prompting users to reflect on the nature and intensity of their reliance on AI-mediated communication.

The application is structured as a Flask web server that exposes a REST API endpoint. The frontend, built with HTML, CSS, and JavaScript, provides a clean interface where users submit text and receive an immediate risk assessment accompanied by a confidence score, a probability breakdown across all three categories, and personalized suggestions relevant to the predicted risk level.

2. Methodology

The development of ERRAM followed a structured, iterative approach that moved from requirement analysis through to testing and deployment readiness. Each phase produced concrete deliverables that informed the next stage of development.

2.1 Requirement Analysis

The initial phase involved identifying the core problem: the absence of any mechanism within conversational AI platforms to detect or communicate emotional risk to users. The key functional requirements established during this phase were as follows:

- A text input interface accessible through a standard web browser without requiring any installation on the user's device.
- A backend capable of processing natural language text and returning a risk classification within a response time acceptable for interactive use.
- Three distinct risk categories - Low, Medium, and High with descriptive labels and actionable recommendations for each.
- A confidence score accompanying every prediction to communicate the model's certainty to the user.
- A probability breakdown showing the model's computed likelihood for each of the three categories.
- A fallback dataset to ensure the application functions even in the absence of the external Kaggle files.

2.2 System Design

System design addressed both the machine learning pipeline and the web application architecture. The decision to use TF-IDF combined with Logistic Regression was made deliberately: this combination is well-understood, computationally lightweight, and produces interpretable outputs without the opacity associated with deep neural network approaches. The web application was designed around a client-server model with a clean separation between the prediction logic and the presentation layer.

The emotion-to-risk mapping was designed based on the clinical and psychological associations of each emotion label in the Kaggle dataset. Sadness and fear, which are strongly associated with emotional distress and vulnerability, were mapped to High risk. Joy and love, reflecting positive emotional states, were assigned to Low risk. Surprise and anger, which represent emotional activation without clear directional valence, were placed in the Medium risk category.

2.3 Development

Development was carried out collaboratively across two Git branches, with each contributor responsible for distinct components of the system. Kritika Bansal developed the data loading module, text preprocessing function, and model training pipeline. Niharika Kapoor implemented the prediction function, Flask API routes, and the complete frontend interface including HTML templates, CSS styling, and JavaScript interaction logic.

The machine learning pipeline was implemented as a scikit-learn Pipeline object combining a TfidfVectorizer configured with unigram and bigram extraction, a maximum feature limit of 8,000 terms, and sublinear term frequency scaling, followed by a Logistic Regression classifier using the lbfgs solver. The trained pipeline was serialized using Python's pickle module and stored as `erram_model.pkl`, which is loaded at application startup to avoid retraining on every request.

2.4 Integration

Integration connected the machine learning model to the web interface through a Flask REST API. The `/predict` endpoint receives a JSON payload containing the user's text, passes it through the prediction pipeline, and returns a structured JSON response. The frontend JavaScript function `analyzeRisk()` sends an asynchronous POST request to this endpoint using the browser's native Fetch API, then updates the DOM dynamically based on the response without requiring a page reload.

2.5 Testing and Validation

Testing was performed at multiple levels. The model was evaluated on the held-out validation split from the Kaggle dataset, and separately on the test.txt file where available. Manual testing covered a range of input types including clearly positive statements, statements of moderate distress, high-risk expressions, and edge cases such as academic references to mental health terminology and short inputs.

2.6 Deployment Readiness

The application is already deployed and available at emotional-risk-detector-ai-1.onrender.com/. It is also structured for local deployment on any machine meeting the hardware and software requirements in Section 2. Running python app.py starts the Flask development server locally, and the model trains automatically on first run if no serialized model file is detected. For production use, the app can be hosted on platforms like Render with minimal configuration changes.

3. Tools and Technologies

The development of ERRAM involved a combination of backend, frontend, and machine learning technologies to ensure efficient processing, usability, and performance.

3.1 Programming Language

- Python 3.10

3.2 Backend Framework

- Flask (for API development and server-side logic)

3.3 Machine Learning & NLP

- scikit-learn
 - TF-IDF Vectorizer
 - Logistic Regression
- pandas (data handling)
- NumPy (numerical operations)

3.4 Frontend Technologies

- HTML5 (structure)
- CSS3 (styling and layout)
- JavaScript (dynamic interaction and API calls)

3.5 Dataset

- Kaggle Emotions Dataset for NLP
 - ~16,000 labelled sentences
 - Emotions mapped to risk levels

3.6 Development Tools

- Git & GitHub (version control and collaboration)
- Visual Studio Code (code editor)

3.7 Model Handling

- Pickle (model serialization and loading)

4. Implementation

4.1 Frameworks and Technologies

Category	Technology Used
Backend Framework	Python 3 with Flask 3.0.3
Machine Learning	scikit-learn 1.5.1 (TfidfVectorizer, LogisticRegression, Pipeline)
Data Handling	pandas 2.2.2, NumPy 1.26.4
Model Persistence	Python pickle module (serializes trained Pipeline to .pkl file)
Frontend Markup	HTML5 with Jinja2 templating via Flask
Frontend Styling	CSS3 with custom properties (variables) and Google Fonts
Frontend Logic	Vanilla JavaScript with Fetch API for async HTTP requests
Version Control	Git with GitHub (two branches: kritika and niharika)
Dataset	Kaggle Emotions Dataset for NLP (16,000+ labelled sentences)

4.2 Core Logic

- **User Input:** The textarea element in index.html captures text entry. A JavaScript event listener counts characters in real time and displays the count below the textarea. The Ctrl+Enter keyboard shortcut triggers submission as an alternative to the button click.
- **Text Cleaning:** The clean_text() function in train_model.py processes every input both during model training and during live prediction through the same sequence: lowercasing, punctuation removal via regex substitution, and whitespace normalisation. This consistency is critical; any discrepancy between training and inference preprocessing would degrade model performance.
- **TF-IDF Feature Extraction:** The TfidfVectorizer is configured with ngram_range=(1,2) to capture both single words and two-word phrases, max_features=8000 to limit vocabulary size,

- `stop_words='english'` to exclude common function words, and `sublinear_tf=True` to apply logarithmic scaling to term frequencies. This configuration balances expressiveness with computational efficiency.
- **Model Training:** `train_and_save_model()` loads the Kaggle dataset, maps emotion labels to risk codes using `EMOTION_TO_RISK`, applies `clean_text()` to all sentences, performs a stratified 85/15 train-validation split using `random_state=42` for reproducibility, fits the Pipeline, and prints a classification report. The trained pipeline is saved to `model/erram_model.pkl` using `pickle.dump()`.
- **Risk Prediction:** `predict_risk()` loads the saved model using `pickle.load()`, applies `clean_text()` to the user's input, calls `pipeline.predict()` for the class label and `pipeline.predict_proba()` for probability scores, computes the confidence as the maximum probability value, and assembles the response dictionary containing the risk level, colour code, description, tips, and full probability breakdown.
- **Result Presentation:** The `displayResult()` JavaScript function receives the JSON response and updates six elements: the risk badge text and CSS class, the confidence bar width and percentage label, the probability bar widths for all three categories, the description paragraph, the tips list, and optionally the warning box if the API returns a warning field.

4.3 Emotion to Risk Mapping

The Kaggle dataset labels each sentence with one of six emotions. These are mapped to risk levels during training data preparation as follows:

Kaggle Emotion	Risk Level	Rationale
joy, love	Low (0)	Positive emotional states indicating healthy engagement
surprise	Medium (1)	Emotionally activated but directionally ambiguous
anger	Medium (1)	Reactive emotion, not necessarily indicative of deep distress
sadness, fear	High (2)	Strongly associated with vulnerability and emotional distress

5. Outcomes

The following outcomes were achieved upon completion of the project:

- A fully functional web application that classifies emotional risk from free-text input in real time, accessible through any modern browser on the local network. Here is the deployed link emotional-risk-detector-ai-1.onrender.com/.
- A trained Logistic Regression classifier achieving approximately 96 percent accuracy on validation data, with consistently correct classification of clearly Low and clearly High risk inputs.
- An interpretable result display showing the predicted risk level, model confidence percentage, animated probability bars for all three categories, a contextual description, and three personalised suggestion.

A well-structured Git repository with separate contribution branches for each team member, demonstrating collaborative development practice across both backend and frontend components.

- An About page documenting the project background, dataset, technology stack, team contributions, and known model limitations, suitable for academic submission and evaluation.
- Successful handling of edge case inputs: academic references to emotional terminology are generally classified as Low or Medium risk rather than High, and short positive statements consistently receive Low risk predictions.

5.1 Model Accuracy

The model was evaluated on the held-out validation split (15 percent of the combined training and validation data) and on the separate test.txt file from the Kaggle dataset. Results are summarised below:

Metric	Validation Set	Test Set
Overall Accuracy	~96.04 %	~96.55%
Low Risk F1 Score	High	High

Medium Risk F1 Score	Moderate	Moderate
High Risk F1 Score	High	High

The Medium risk category shows the lowest F1 score, which is consistent with expectations: Medium risk sentences are inherently more ambiguous than clearly positive or clearly distressed statements, and the boundary between Medium and adjacent categories is less distinct in natural language.

Test Input	Expected	Observed
I had a great day today, feeling really positive	Low	Low
Spent quality time with family, life is good	Low	Low
I have been feeling lonely and stressed lately	Medium	Medium
I get anxious when meeting new people	Medium	Medium
I don't want to live this life anymore	High	High
Nobody would care if I disappeared	High	High
I feel completely hopeless about everything	High	High
I studied depression for my psychology exam	Low/Medium	Low
I am dying of laughter right now	Low	Low

6. Conclusion

ERRAM demonstrates that meaningful emotional risk assessment from conversational text is achievable using accessible, well-established machine learning techniques without reliance on large language models or cloud-based inference infrastructure. The combination of TF-IDF feature extraction and Logistic Regression, trained on a labelled emotions corpus, provides a computationally efficient and interpretable classification system that performs reliably across the range of inputs representative of real user interactions with conversational AI platforms.

The project fulfils each of its stated objectives: the web interface is functional and user-friendly, the model achieves competitive accuracy for a lightweight approach, the output is interpretable rather than opaque, and the system runs without specialized hardware. The collaborative development process, structured through Git branching with distinct contributions from each team member across backend and frontend components, reflects professional software engineering practice.

The known limitations of the system principally around sarcasm detection and context-dependent language are accurately documented and are consistent with the inherent constraints of bag-of-words feature extraction methods. These limitations do not diminish the system's utility for its intended purpose but do indicate clear directions for future improvement, particularly through the adoption of transformer-based contextual models.

ERRAM contributes a practical, deployable tool to the emerging field of responsible AI usage monitoring and provides a foundation upon which more sophisticated emotional health awareness systems can be developed.

Future Scope

While ERRAM successfully achieves its objective of emotional risk classification, several enhancements can further improve its effectiveness and applicability.

- **Advanced Models:** Integration of transformer-based models (such as BERT) to improve contextual understanding and handle complex language patterns like sarcasm and ambiguity.
- **Multilingual Support:** Extending the system to support multiple languages for wider accessibility.
- **Real-time Monitoring:** Continuous tracking of user inputs over time to detect patterns of emotional dependency rather than single-instance analysis.
- **Integration with Chat Systems:** Embedding ERRAM within conversational AI platforms to provide real-time feedback during interactions.
- **Personalization:** Adapting suggestions based on user history and behaviour for more meaningful guidance.
- **Mental Health Support Integration:** Connecting high-risk outputs to helplines or professional resources for immediate assistance.
- **Mobile Application Development:** Creating a mobile-friendly or standalone app version for better accessibility.

References

1. Praveengovi. Emotions Dataset for NLP. Kaggle Platform, 2020. Dataset available at: <https://www.kaggle.com/datasets/praveengovi/emotions-dataset-for-nlp>
2. Pedregosa, F., Varoquaux, G., Gramfort, A., et al. Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, Volume 12, pages 2825 to 2830, 2011. Available at: <https://scikit-learn.org>
3. Pallets Projects. Flask Documentation, Version 3.0. 2024. Available at: <https://flask.palletsprojects.com>
4. Python Software Foundation. Python Language Reference, Version 3.10+. Available at: <https://www.python.org>
5. Salton, G. and Buckley, C. Term-Weighting Approaches in Automatic Text Retrieval. *Information Processing and Management*, Volume 24, Issue 5, pages 513 to 523, 1988.
6. Bird, S., Klein, E., and Loper, E. *Natural Language Processing with Python: Analyzing Text with the Natural Language Toolkit*. O'Reilly Media, 2009.
7. Hastie, T., Tibshirani, R., and Friedman, J. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. Second Edition. Springer, 2009.
8. World Health Organisation. *Mental Health: Strengthening Our Response*. WHO Fact Sheet, 2022. Available at: <https://www.who.int/news-room/fact-sheets/detail/mental-health-strengthening-our-response>
9. iCall Psychosocial Helpline. Tata Institute of Social Sciences, Mumbai, India. Available at: <https://icallhelpline.org>

Appendices

Appendix A: Sample Inputs and Outputs

Input Text	Predicted Risk
"I had a great day today"	Low
"I feel stressed and overwhelmed"	Medium
"I don't want to live anymore"	High

Appendix B: Emotion to Risk Mapping

Emotion	Risk Level
Joy, Love	Low
Surprise, Anger	Medium
Sadness, Fear	High

Appendix C: Deployment Link

- Web Application: <https://emotional-risk-detector-ai-1.onrender.com>
- GitHub Repository: github.com/kritkat11/Emotional-Reliance-Risk-Assessment-Model-2210990524-2210991973